

M14 — Representação Textual: BoW, n-gramas e TF-IDF

Aula 4 — Linguagem Natural

Pergunta norteadora

Como representar texto computacionalmente sem perder de vista a pergunta do projeto?

Fronteiras desta aula

- trabalhar **somente com dados de treino** durante o desenvolvimento
- manter a partição de **teste protegida**
- preservar o **corpus bruto**
- distinguir **representação** de **modelo**
- não treinar classificadores nesta aula
- não calcular acurácia, precisão, recall, F1, matriz de confusão ou qualquer outra métrica de desempenho
- não escolher características usando o rótulo

Versão 1.2 — simplificação de carga cognitiva nos Passos 10 e 13, sem alteração da lógica metodológica.

Identificação

Preencha antes de começar:

- **Turma:**
- **Grupo:**
- **Integrantes:**
- **Projeto:** P01 / P02 / P03
- **Pergunta operacional da M05:**

O notebook faz parte do projeto da disciplina. Ele documenta como o grupo transforma os textos em representações computacionais antes da etapa de modelagem.

```
In [1]: import re
import unicodedata
from pathlib import Path

import numpy as np
import pandas as pd
```

```
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer

print("Bibliotecas carregadas.")
```

Bibliotecas carregadas.

Bloco A — Laboratório conceitual

Antes de representar o corpus do projeto, vamos observar o mecanismo com apenas **três documentos pequenos**.

A meta deste bloco é compreender:

texto → vocabulário → vetor → matriz → representação

Passo 1 — Três documentos

Considere:

- **D1:** não gostei do produto
- **D2:** gostei muito do produto
- **D3:** não gostei do atendimento

Conceitos

- **Documento:** uma unidade textual da coleção
- **Coleção:** conjunto de documentos
- **Representação:** forma computacional de descrever o conteúdo textual

Pergunta: como transformar esses textos em algo que um algoritmo possa manipular?

```
In [2]: textos_demo = [
        "não gostei do produto",
        "gostei muito do produto",
        "não gostei do atendimento"
    ]

demo = pd.DataFrame({
    "documento": ["D1", "D2", "D3"],
    "texto": textos_demo
})

display(demo)
```

	documento	texto
0	D1	não gostei do produto
1	D2	gostei muito do produto
2	D3	não gostei do atendimento

Passo 2 — Bag of Words e vocabulário

Conceitos

- **Termo:** unidade usada como característica da representação
- **Vocabulário:** conjunto de termos distintos considerados na coleção
- **Característica (feature):** dimensão usada para descrever um documento
- **Vetor:** sequência numérica que representa um documento
- **Bag of Words (BoW):** representação baseada na ocorrência ou frequência dos termos

Neste primeiro exemplo, cada termo do vocabulário se tornará uma característica.

Nota sobre `fit_transform()`: neste notebook, `fit` significa aprender o vocabulário e as estatísticas da representação a partir dos textos. Isso **não** significa treinar um classificador.

```
In [3]: bow_demo = CountVectorizer()
X_bow_demo = bow_demo.fit_transform(textos_demo)

vocab_demo = bow_demo.get_feature_names_out()

print("Documentos:", X_bow_demo.shape[0])
print("Vocabulário:", X_bow_demo.shape[1], "termos")
print("\nTermos:")
for termo in vocab_demo:
    print("-", termo)
```

Documentos: 3
Vocabulário: 6 termos

Termos:
- atendimento
- do
- gostei
- muito
- não
- produto

Passo 3 — Matriz documento-termo

Na matriz documento-termo:

- **linhas** = documentos
- **colunas** = termos/características
- **valor** = frequência do termo naquele documento

Perguntas de leitura:

1. O que representa cada linha?

2. O que representa cada coluna?
3. O que significam 0 e 1?
4. Onde ficou a ordem das palavras?

```
In [4]: matriz_bow_demo = pd.DataFrame(
        X_bow_demo.toarray(),
        columns=vocab_demo,
        index=["D1", "D2", "D3"]
    )

display(matriz_bow_demo)
```

	atendimento	do	gostei	muito	não	produto
D1	0	1	1	0	1	1
D2	0	1	1	1	0	1
D3	1	1	1	0	1	0

Passo 4 — Dimensionalidade e esparsidade

- **Dimensionalidade:** quantidade de características usadas para representar cada documento
- **Esparsidade:** predominância de valores zero na matriz

Matrizes de texto costumam ser esparsas porque cada documento contém apenas uma pequena parte do vocabulário total.

```
In [5]: total_posicoes = X_bow_demo.shape[0] * X_bow_demo.shape[1]
        nao_zero = X_bow_demo.nnz
        zeros = total_posicoes - nao_zero
        esparsidade = zeros / total_posicoes if total_posicoes else 0

        print("Documentos:", X_bow_demo.shape[0])
        print("Características:", X_bow_demo.shape[1])
        print("Posições totais:", total_posicoes)
        print("Valores diferentes de zero:", nao_zero)
        print("Zeros:", zeros)
        print(f"Esparsidade: {esparsidade:.1%}")
```

```
Documentos: 3
Características: 6
Posições totais: 18
Valores diferentes de zero: 12
Zeros: 6
Esparsidade: 33.3%
```

Passo 5 — Unigramas e bigramas

Compare:

- não gostei
- gostei não

Com unigramas, os mesmos termos podem estar presentes mesmo quando a ordem local muda.

Conceitos

- **Unigrama:** sequência de 1 token
- **Bigrama:** sequência de 2 tokens consecutivos
- **n-grama:** sequência de `n` tokens consecutivos

Bigramas preservam um pouco mais de **contexto local**, mas aumentam a dimensionalidade.

```
In [6]: bow_uni_bi_demo = CountVectorizer(ngram_range=(1, 2))
X_uni_bi_demo = bow_uni_bi_demo.fit_transform(textos_demo)

features_uni_bi = bow_uni_bi_demo.get_feature_names_out()
bigramas_demo = [f for f in features_uni_bi if " " in f]

print("Somente unigramas:", X_bow_demo.shape[1], "características")
print("Unigramas + bigramas:", X_uni_bi_demo.shape[1], "características")
print("\nBigramas encontrados:")
for bg in bigramas_demo:
    print("-", bg)
```

Somente unigramas: 6 características
Unigramas + bigramas: 12 características

Bigramas encontrados:

- do atendimento
- do produto
- gostei do
- gostei muito
- muito do
- não gostei

Passo 6 — TF-IDF

A contagem simples responde: **quantas vezes o termo aparece?**

O TF-IDF acrescenta outra ideia: **o quanto esse termo ajuda a caracterizar um documento em relação à coleção?**

- **TF — Term Frequency:** frequência do termo no documento
- **DF — Document Frequency:** número de documentos em que o termo aparece
- **IDF — Inverse Document Frequency:** reduz o destaque de termos muito difundidos na coleção
- **TF-IDF:** combina importância local e distribuição do termo na coleção

Não estamos medindo desempenho de modelo. Estamos apenas mudando a forma de representar os textos.

```
In [7]: tfidf_demo = TfidfVectorizer()
X_tfidf_demo = tfidf_demo.fit_transform(textos_demo)

matriz_tfidf_demo = pd.DataFrame(
    X_tfidf_demo.toarray(),
    columns=tfidf_demo.get_feature_names_out(),
    index=["D1", "D2", "D3"]
)

display(matriz_tfidf_demo.round(3))
```

	atendimento	do	gostei	muito	não	produto
D1	0.000	0.434	0.434	0.000	0.558	0.558
D2	0.000	0.391	0.391	0.663	0.000	0.504
D3	0.663	0.391	0.391	0.000	0.504	0.000

Interprete antes de avançar

Discuta:

1. Por que os valores do TF-IDF não são apenas 0 e 1?
2. Que termos receberam maior destaque em alguns documentos?
3. O que TF-IDF representa de maneira diferente da contagem simples?
4. Qual aspecto da ordem das palavras continua pouco representado?

Finalizado o laboratório conceitual, passamos agora ao corpus real do grupo.

Bloco B — Represente o corpus do seu projeto

A partir daqui, trabalhe com o **mesmo P01, P02 ou P03** definido na M05.

Todas as decisões de desenvolvimento devem usar **somente a partição de treino**.

O teste permanece protegido.

Passo 7 — Configure o projeto

Altere **somente** o valor de PROJETO.

Opções:

- P01 — reclamações por assunto
- P02 — sentimento em avaliações
- P03 — solicitações por setor

```
In [8]: PROJETO = "P03" # ALTERE SOMENTE ESTA LINHA: P01, P02 ou P03

CONFIG = {
    "P01": {"arquivo": "P01_reclamacoes_assunto.csv", "texto": "texto", "
    "P02": {"arquivo": "P02_sentimento_avaliacoes.csv", "texto": "texto",
    "P03": {"arquivo": "P03_solicitacoes_atendimento.csv", "texto": "text
}

if PROJETO not in CONFIG:
    raise ValueError("PROJETO deve ser P01, P02 ou P03.")

config = CONFIG[PROJETO]
ARQUIVO = config["arquivo"]
CAMPO_TEXTO = config["texto"]
ROTULO = config["rotulo"]

print("Projeto:", PROJETO)
print("Arquivo:", ARQUIVO)
print("Texto:", CAMPO_TEXTO)
print("Saída:", ROTULO)
```

```
Projeto: P03
Arquivo: P03_solicitacoes_atendimento.csv
Texto: texto
Saída: setor_destino
```

Passo 8 — Carregue e proteja o corpus

Neste passo:

1. carregamos o CSV oficial
2. preservamos uma cópia do corpus bruto em memória
3. selecionamos **somente treino**
4. retiramos da área de representação os registros sem texto

Isso **não altera o CSV original**.

Registros sem texto não podem ser representados linguisticamente, mas o corpus bruto permanece preservado para rastreabilidade.

```
In [9]: caminho = Path(ARQUIVO)
```

```

if not caminho.exists():
    try:
        from google.colab import files
        print(f"Arquivo esperado: {ARQUIVO}")
        files.upload()
    except ImportError:
        pass

if not Path(ARQUIVO).exists():
    raise FileNotFoundError(
        f"Arquivo '{ARQUIVO}' não encontrado. Envie o CSV correspondente
    )

df = pd.read_csv(ARQUIVO, sep=";", encoding="utf-8-sig")
df_bruto = df.copy(deep=True)

if "particao_recomendada" not in df.columns:
    raise ValueError("Campo 'particao_recomendada' não encontrado no CSV.")

df_treino = df.loc[df["particao_recomendada"].eq("treino")].copy()

df_treino_valido = df_treino.loc[
    df_treino[CAMPO_TEXTO].notna()
    & df_treino[CAMPO_TEXTO].astype(str).str.strip().ne("")
].copy()

print("Corpus completo:", len(df), "registros")
print("Treino:", len(df_treino), "registros")
print("Treino com texto disponível:", len(df_treino_valido), "registros")
print("Teste utilizado para desenvolvimento: NÃO")

```

Corpus completo: 5000 registros

Treino: 3500 registros

Treino com texto disponível: 3470 registros

Teste utilizado para desenvolvimento: NÃO

Passo 9 — Recupere a decisão de preparação da Aula 3

Configure as opções abaixo de acordo com a decisão registrada no M10.

Não existe uma preparação automaticamente correta para todos os projetos.

Nesta aula:

- a normalização mínima é sempre aplicada
- minúsculas podem ser ativadas ou não
- stopwords podem ser removidas ou não
- negações podem ser preservadas quando a remoção de stopwords estiver ativa
- stemming pode ser ativado ou não
- **lematização não será aplicada como etapa padrão do corpus**

- pontuação e emojis são preservados quando a remoção de stopwords não está ativa
- quando a remoção de stopwords está ativa, o filtro experimental mantém apenas tokens alfanuméricos, como no M10

```
In [10]: # CONFIGURE DE ACORDO COM A DECISÃO DO M10
USAR_MINUSCULAS = True
REMOVER_STOPWORDS = False
PRESERVAR_NEGACAO = True
USAR_STEMMING = False

decisoões_preparacao = pd.DataFrame({
    "decisão": ["usar minúsculas", "remover stopwords", "preservar negaçã",
    "configuração": [
        "SIM" if USAR_MINUSCULAS else "NÃO",
        "SIM" if REMOVER_STOPWORDS else "NÃO",
        "SIM" if PRESERVAR_NEGACAO else "NÃO",
        "SIM" if USAR_STEMMING else "NÃO"
    ]
})

display(decisoões_preparacao)
```

	decisão	configuração
0	usar minúsculas	SIM
1	remover stopwords	NÃO
2	preservar negação	SIM
3	usar stemming	NÃO

Passo 10 — Produza texto original x preparado

Agora a decisão da Aula 3 será aplicada aos textos de treino.

Vamos criar duas versões:

- `texto_original`
- `texto_preparado`

O original permanece preservado.

A comparação serve para verificar se a transformação aplicada corresponde ao que o grupo decidiu no M10.

Passo 10A — Infraestrutura da preparação

Execute esta célula sem alterar. Ela reúne recursos técnicos necessários para aplicar, no Passo 10B, as decisões registradas

no Passo 9.

```
In [11]: # Normalização mínima usada antes das decisões de preparação
def normalizacao_minima(texto):
    texto = unicodedata.normalize("NFC", str(texto))
    texto = re.sub(r"\s+", " ", texto).strip()
    return texto

# Stopwords: usa NLTK se o recurso já estiver disponível.
# Caso contrário, aplica imediatamente uma lista local de fallback.
try:
    from nltk.corpus import stopwords
    STOPWORDS_PT = set(stopwords.words("portuguese"))
except Exception:
    STOPWORDS_PT = {
        "a", "o", "os", "as", "de", "da", "do", "das", "dos", "e", "é", "em", "um", "u",
        "para", "por", "com", "que", "se", "na", "no", "nas", "nos", "ao", "aos", "à",
        "eu", "tu", "ele", "ela", "eles", "elas", "me", "te", "nos", "vos", "lhe", "e",
        "meu", "minha", "meus", "minhas", "seu", "sua", "seus", "suas", "este", "e",
        "esse", "essa", "isto", "isso", "aquilo", "como", "mais", "menos", "muito"
    }

# As negações ficam explicitamente protegidas quando PRESERVAR_NEGACAO =
NEGACOES = {"não", "nao", "nem", "nunca", "jamais"}

# Stemmer só é criado quando a decisão do M10 solicitar stemming.
stemmer = None
if USAR_STEMMING:
    try:
        from nltk.stem.snowball import SnowballStemmer
        stemmer = SnowballStemmer("portuguese")
    except Exception as e:
        raise RuntimeError("Não foi possível iniciar o SnowballStemmer.")

# Padrão alinhado ao M10: preserva palavras e também sinais/emoji como to
PADRAO_TOKEN = re.compile(
    r"\w+(?:[-']\w+)*|[\^\w\s]",
    flags=re.UNICODE
)

print("Infraestrutura da preparação pronta.")
```

Infraestrutura da preparação pronta.

Passo 10B — Aplique as decisões do M10

Leia esta célula como uma sequência de decisões: **normalizar** → **usar minúsculas?** → **remover stopwords?** → **preservar negação?** → **usar stemming?**

```
In [12]: # Aplica ao texto somente as decisões ativadas no Passo 9.
def preparar_texto(texto):
    t = normalizacao_minima(texto)

    # 1) Minúsculas
    if USAR_MINUSCULAS:
        t = t.lower()
```

```

# 2) Tokenização
tokens = PADRAO_TOKEN.findall(t)

# 3) Stopwords, se essa decisão estiver ativada
if REMOVE_STOPWORDS:
    stopset = set(STOPWORDS_PT)
    if PRESERVAR_NEGACAO:
        stopset = stopset - NEGACOES

    tokens = [
        tok for tok in tokens
        if any(ch.isalnum() for ch in tok) and tok not in stopset
    ]

# 4) Stemming, se essa decisão estiver ativada
if USAR_STEMMING:
    tokens = [
        stemmer.stem(tok) if any(ch.isalnum() for ch in tok) else tok
        for tok in tokens
    ]

return " ".join(tokens)

# Cria uma área de trabalho sem alterar o corpus bruto.
trabalho = df_treino_valido.copy()
trabalho["texto_original"] = trabalho[CAMPO_TEXTO].astype(str)
trabalho["texto_preparado"] = trabalho["texto_original"].map(preparar_texto)

# Mostra exemplos para comparar original x preparado.
display(
    trabalho[["id", "texto_original", "texto_preparado"]]
    .head(3)
)

```

	id	texto_original	texto_preparado
0	ATE-00001	Por favor, estou com uma dúvida, tenho dúvida ...	por favor , estou com uma dúvida , tenho dúvid...
1	ATE-00002	Oi, preciso consultar uma pendência de emprést...	oi , preciso consultar uma pendência de emprés...
3	ATE-00004	aparece uma mensagem de erro toda vez que faço...	aparece uma mensagem de erro toda vez que faço...

Passo 11 — Compare BoW no original x preparado

Agora podemos observar uma consequência quantitativa da preparação.

Pergunta:

a preparação alterou o vocabulário e a dimensionalidade da representação?

Importante:

menor vocabulário ou menor dimensionalidade não significam automaticamente melhor representação.

Os vetorizadores desta etapa usam `lowercase=False`. Assim, a decisão sobre converter ou não para minúsculas permanece sob controle da preparação definida pelo grupo, e não do `CountVectorizer`.

```
In [13]: bow_original = CountVectorizer(lowercase=False)
X_bow_original = bow_original.fit_transform(trabalho["texto_original"])

bow_preparado = CountVectorizer(lowercase=False)
X_bow_preparado = bow_preparado.fit_transform(trabalho["texto_preparado"])

comparacao_bow = pd.DataFrame({
    "versão": ["original", "preparado"],
    "documentos": [X_bow_original.shape[0], X_bow_preparado.shape[0]],
    "características": [X_bow_original.shape[1], X_bow_preparado.shape[1]]
})

display(comparacao_bow)
print("Diferença no número de características:", X_bow_preparado.shape[1]
```

	versão	documentos	características
0	original	3470	557
1	preparado	3470	288

Diferença no número de características: -269

Passo 12 — Compare unigramas × unigramas + bigramas

Pergunta:

quanto contexto local é acrescentado e quanto cresce a representação?

Vamos comparar:

- `(1, 1)` → somente unigramas
- `(1, 2)` → unigramas + bigramas

Os exemplos exibidos serão retirados apenas do **treino** e não serão selecionados com base no rótulo.

```
In [14]: vec_uni = CountVectorizer(ngram_range=(1, 1), lowercase=False)
X_uni = vec_uni.fit_transform(trabalho["texto_preparado"])

vec_uni_bi = CountVectorizer(ngram_range=(1, 2), lowercase=False)
X_uni_bi = vec_uni_bi.fit_transform(trabalho["texto_preparado"])

comparacao_ngramas = pd.DataFrame({
```

```

    "representação": ["unigramas", "unigramas + bigramas"],
    "características": [X_uni.shape[1], X_uni_bi.shape[1]]
})

display(comparacao_ngramas)
features = vec_uni_bi.get_feature_names_out()
bigramas = [f for f in features if " " in f]
print("\nAté 10 bigramas do vocabulário de treino:")
for bg in bigramas[:10]:
    print("-", bg)

```

	representação	características
0	unigramas	288
1	unigramas + bigramas	2278

Até 10 bigramas do vocabulário de treino:

- aberto agradeço
- aberto além
- aberto antes
- aberto como
- aberto conseguem
- aberto desde
- aberto em
- aberto fico
- aberto isso
- aberto mas

Passo 13 — TF-IDF no corpus

Agora construiremos TF-IDF sobre `texto_preparado`.

Pergunta:

que termos recebem maior destaque dentro de alguns documentos de treino?

Atenção:

não estamos procurando os “melhores termos para prever a classe”.

Estamos apenas interpretando como cada documento é representado.

```

In [15]: # Cria a representação TF-IDF usando apenas os textos preparados de trein
tfidf = TfidfVectorizer(lowercase=False)
X_tfidf = tfidf.fit_transform(trabalho["texto_preparado"])
termos_tfidf = tfidf.get_feature_names_out()

print("Documentos:", X_tfidf.shape[0])
print("Características:", X_tfidf.shape[1])
print("Matriz:", X_tfidf.shape)

# Inspecciona somente os três primeiros documentos, sem converter a matriz
# Em caso de empate, ordena alfabeticamente pelo termo para manter a saída
linhas_saida = []
for pos in range(min(3, X_tfidf.shape[0])):

```

```

vetor = X_tfidf.getrow(pos)

pesos_doc = pd.DataFrame({
    "termo": termos_tfidf[vetor.indices],
    "peso_tfidf": vetor.data
})

top5 = (
    pesos_doc
    .sort_values(
        by=["peso_tfidf", "termo"],
        ascending=[False, True]
    )
    .head(5)
)

for _, linha in top5.iterrows():
    linhas_saida.append({
        "id": trabalho.iloc[pos]["id"],
        "termo": linha["termo"],
        "peso_tfidf": round(float(linha["peso_tfidf"]), 3)
    })

display(pd.DataFrame(linhas_saida))

```

Documentos: 3470

Características: 288

Matriz: (3470, 288)

	id	termo	peso_tfidf
0	ATE-00001	dúvida	0.421
1	ATE-00001	que	0.320
2	ATE-00001	funciona	0.281
3	ATE-00001	estou	0.262
4	ATE-00001	retorno	0.252
5	ATE-00002	aberto	0.290
6	ATE-00002	comprovante	0.290
7	ATE-00002	débito	0.290
8	ATE-00002	enviado	0.290
9	ATE-00002	consultar	0.284
10	ATE-00004	faço	0.393
11	ATE-00004	mensagem	0.393
12	ATE-00004	toda	0.393
13	ATE-00004	vez	0.393
14	ATE-00004	login	0.335

Passo 14 — Compare as representações

Agora reúna as evidências.

Pergunta:

o que cada representação tornou disponível e qual custo introduziu?

Nenhuma representação será declarada automaticamente “vencedora”.

```
In [16]: resumo_representacoes = pd.DataFrame({
    "representação": ["BoW original", "BoW preparado", "BoW unigramas + b",
    "documentos": [X_bow_original.shape[0], X_bow_preparado.shape[0], X_u
    "características": [X_bow_original.shape[1], X_bow_preparado.shape[1]
    })

display(resumo_representacoes)

quadro_conceitual = pd.DataFrame({
    "representação": ["BoW", "unigramas + bigramas", "TF-IDF"],
    "preserva principalmente": ["ocorrência/frequência dos termos", "ocor
    "limitação principal": ["perde grande parte da ordem", "aumenta a dim
    })

display(quadro_conceitual)
```

	representação	documentos	características
0	BoW original	3470	557
1	BoW preparado	3470	288
2	BoW unigramas + bigramas	3470	2278
3	TF-IDF preparado	3470	288

	representação	preserva principalmente	limitação principal
0	BoW	ocorrência/frequência dos termos	perde grande parte da ordem
1	unigramas + bigramas	ocorrência + algum contexto local	aumenta a dimensionalidade e ainda preserva co...
2	TF-IDF	importância relativa dos termos	perde grande parte da ordem e depende do vocab...

Passo 15 — Decisão inicial de representação

Preencha com base **nas evidências observadas neste notebook**.

Decisão inicial

Representação que parece mais promissora:

...

Evidência observada:

...

Principal vantagem:

...

Principal limitação:

...

O que ainda precisa ser comparado na próxima aula:

...

Esta decisão é inicial. Ainda não treinamos modelos e, portanto, ainda não sabemos qual representação produzirá melhor desempenho preditivo.

```
In [17]: corpus_preservado = df.equals(df_bruto)
so_treino = bool(trabalho["particao_recomendada"].eq("treino").all())

print("Corpus bruto preservado:", "OK" if corpus_preservado else "REVISAR")
print("Desenvolvimento realizado apenas com treino:", "OK" if so_treino else "REVISAR")
print("Conteúdo do teste usado para ajustes: NÃO")
print("BoW produzido:", "OK" if X_bow_preparado.shape[0] > 0 else "REVISAR")
print("n-gramas produzidos:", "OK" if X_uni_bi.shape[0] > 0 else "REVISAR")
print("TF-IDF produzido:", "OK" if X_tfidf.shape[0] > 0 else "REVISAR")

if not corpus_preservado:
    raise AssertionError("O DataFrame bruto foi alterado. Revise as células")
if not so_treino:
    raise AssertionError("Há registros fora da partição de treino na área de teste")
```

```
Corpus bruto preservado: OK
Desenvolvimento realizado apenas com treino: OK
Conteúdo do teste usado para ajustes: NÃO
BoW produzido: OK
n-gramas produzidos: OK
TF-IDF produzido: OK
```